Jev活用の最前線〜実践事例から学ぶ、Jevの強み・課題と活用ノウハウ〜

Jev で実現する Generative UI

azukiazusa

自己紹介

- **azukiazusa**
- Frontend Engineer
- https://azukiazusa.dev
- 週に 1 回、Web 開発と AI の記事を書いています
- FE（フロントエンド | ファイアーエムブレム）が好き



今日のゴール

- **Generative UI** とは何か
- なぜ **Jev** と相性がいいのか

AI は文章だけでなく
UI を返すようになった

数字の羅列より、グラフのほうが理解しやすい

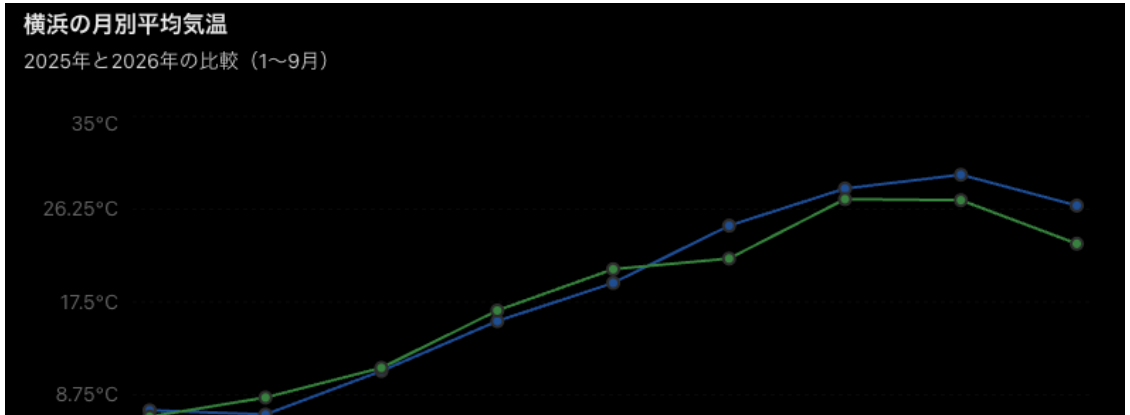
Text

1. 月別平均気温の比較

単位：°C。前年差は「2026年 – 2025年」です。

月	2025年	2026年	前年差
1月	7.3	6.7	-0.6
2月	6.9	8.5	+1.6
3月	11.0	11.3	+0.3
4月	15.7	16.7	+1.0
5月	19.3	20.6	+1.3
6月	24.7	21.6	-3.1
7月	28.2	27.2	-1.0

UI



Claude も ChatGPT も、回答を UI で返し始めた

2026.03

Claude

- 会話の中にチャートや図を表示
- 必要と判断すれば自ら作成

2026.10

ChatGPT

- Intelligent UI
- グラフ、フォーム、小さなツールを回答に表示

Generative UI とは？
AI が文脈に応じて
UI を返すこと

プロダクトに組み込むときの課題

生成された UI を、どうプロダクトの品質に合わせるか？

- 見た目を既存のデザインシステムに揃えたい
- 存在しないコンポーネントや Props を防ぎたい
- 表示するだけでなく、振る舞いも制御したい

デザインシステムのコンポーネントを返す

Design System
コンポーネント
→
AI
画面を組み立てる
→
Product
いつもの画面が AI により出力される

json-render や A2UI といった Generative UI のライブラリは、デザインシステムのコンポーネントを返すことを前提に作られている

これまでは LLM に*ルール*を渡して生成させてきた

- コンポーネントの定義からプロンプトを作る
- 出力形式のルールも一緒に渡す
- LLM はルールに沿って UI を表す文字列を生成する
- アプリ側で検証して描画する

例: json-render は JSON Patch、OpenUI は OpenUI Lang を出力する

出力形式やルールをプロンプトで渡す

```
const catalog = new Catalog({ /* コンポーネントを定義 */ });

const result = streamText({
  model: "openai/gpt-4.1-mini",
  system: catalog.prompt(),
  prompt: buildUserPrompt({ prompt }),
});

return result.toTextStreamResponse();
```

- `catalog.prompt()` にコンポーネントの定義と出力形式の指示が入る
- JSON を書いてもらうには、形式の説明が必要

プロンプトの一例

```
OUTPUT FORMAT (JSONL, RFC 6902 JSON Patch):
Output JSONL (one JSON object per line) ...

RULES:
1. Output ONLY JSONL patches - ... no markdown, no code fences
5. ONLY use components listed above
14. CRITICAL: The "on" field goes on the ELEMENT object, NOT inside "props".
```

- コンポーネントの説明は数行で、大半は出力形式や記法のルール

Jev を使用したアプローチ

UI の組み立てを、Jev の「選択」で実現する

- コンポーネントの候補をアプリ側で用意する
- Jev は候補の中から必要なコンポーネントと配置を選ぶ
- json-render は Jev の選択結果から UI を構成する

コンポーネントを生成させるのではなく、候補の中から選ばせることで、出力形式やルールをプロンプトで渡す必要がなくなる

Jev では Props まで候補として用意する

- LLM に生成させていた Props の値を、あらかじめ候補に設定する
  - 従来のアプローチ: Input コンポーネントのラベルを都度生成
  - Jev: Input コンポーネントのラベルを候補として先に用意する
    - 「メールアドレス」「電話番号」...

Props の値を動的に決めることはできないので、用途の決まった UI を用意する必要がある

コンポーネントの候補を用意する

```
const nameCandidate = {
  id: "name",
  description: 'Editable name, label "名前", bound to /name',
  root: false,
  element: {
    type: "Input",
    props: {
      label: "名前",
      type: "text",
      value: { $bindState: "/name" },
    },
  },
} satisfies Experimental_CompositionCandidate;
```

Jev の選択を元に UI を組み立てる

```
const evaluate = async ({ state, questions, signal }) => {
  const { answers } = await client.systemOne(
    { model: "jev-latest", state, questions },
    { signal },
  );
  return { answers };
};

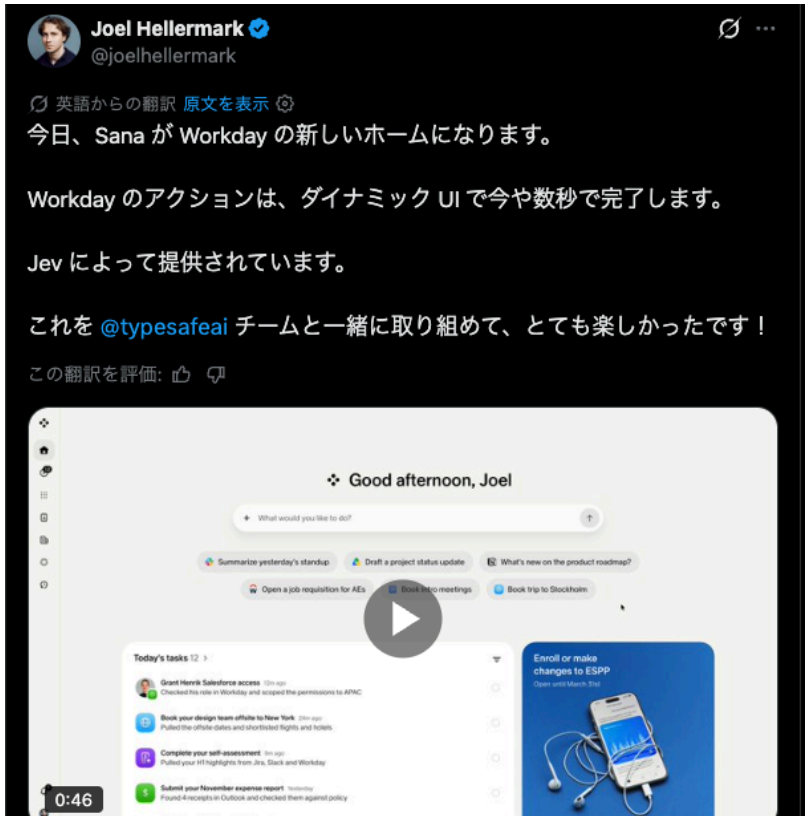
for await (const event of experimental_composeSpec({
  catalog, candidates, initialState, prompt, evaluate,
})) {
  if (event.spec) updatePreview(event.spec);
}
```

Jev と GPT-4.1 mini の比較



事例: Workday の Sana は Jev で UI を組み立てる

- 依頼に合わせて UI が変わる
- フォームやカレンダーを表示
- デザインシステムの全コンポーネントを Jev に渡す



まとめ

1. AI の回答は、文章より **UI** のほうが伝わる場面がある
2. プロダクトでは **デザインシステムのコンポーネント** を返す
3. コンポーネントから選ぶ Generative UI は、選択に特化した **Jev** と相性がいい

参考

- Claude: https://claude.com/blog/claude-builds-visuals
- Intelligent UI: https://techcrunch.com/2026/10/07/chatgpt-is-getting-a-lot-more-visual-with-the-launch-of-a-new-interface/
- Workday: Sana Is the New Workday Home
- The New Stack: https://thenewstack.io/workday-is-using-typesafes-jev-to-build-user-interfaces-on-the-fly/
- json-render: https://json-render.dev/docs/jev
- Jev: https://docs.typesafe.ai/introduction
- json-render と Jev の解説記事: https://azukiazusa.dev/blog/json-render-jev